

APEX brutal integration review

Executive verdict

The biggest missed integration opportunity is **not** “you should have picked a bigger base model.” It is this: **you should have made Granite’s new intrinsic stack the actual trust-and-control plane of APEX.** In the public repo, the surfaces a judge is most likely to inspect still show fallback or swap-point behavior: the OpenRouter chat route has a hard-coded stub mode and only a thin proxy path in production, the Guardian backend file describes the current implementation as a deterministic rule-engine floor with model-level integration deferred, the COA parser still reads a JSON stub instead of a live PDF parse, the timing-sheet parser still returns canned fixture output, and the LangGraph runtime panel explicitly says HEAD is showing a canned fallback while the real runtime is queued. By contrast, Granite Switch 4.1 preview was released in early May 2026 specifically to compose **12** Granite adapters from the RAG, Core, and Guardian libraries, and Mellea is built around instruct-validate-repair loops with validation and repair strategies. That is the missing move that would have turned your weakest, most judge-visible surfaces into your strongest IBM-native story. ¹

What the public repo actually proves

The repo README and architecture docs present a very ambitious IBM stack: Granite-Docling and Docling for COA ingestion, Granite Vision for timing sheets, Granite TimeSeries TTM and FlowState for forecasting, TSPulse for anomalies, Granite Embedding R2 for RAG, Granite Instruct for narration, Granite Guardian for audit, and LangGraph plus Granite MCP Gateway plus ContextForge for orchestration. The same docs also describe Mellea IVR, a tri-agent critic loop, and a Langflow-to-LangGraph handoff. On paper, that is a serious story. ²

What a brutal judge will notice, though, is that the **publicly verifiable runtime** is shallower than the architecture narrative. The COA parser file says the current module reads a Sarah-style COA JSON stub and that the Granite-Docling PDF path is still a swap-point. The timing-sheet parser file says Granite Vision 4.1 4B is the intended production path, but the module currently ships a canned-fixture parse. The `/api/timing-sheet-parse` route repeats that the current HEAD implementation returns canned output and leaves the real Granite Vision or OpenRouter vision path queued. The Guardian module says the actual Granite Guardian model integration lands later and the current code is the deterministic rule floor. The voice-debrief component says HEAD uses browser-native SpeechRecognition and that Watson STT is still a swap-point. The LangGraph runtime panel also says HEAD is still showing a canned fallback while the real LangGraph + MCP + ContextForge runtime is still to be wired. ³

That does **not** mean the project is weak. It means the project is vulnerable to a very specific criticism: **“You built an IBM architecture diagram faster than you built an IBM runtime.”** For a May 31, 2026 challenge submission, that is exactly the kind of attack line a good judge will use. ⁴

The biggest missed integration opportunity

If I have to name **one** missing Granite variant, it is **Granite Switch 4.1-3B preview**. Granite 4.1 itself is already strong across tool calling, instruction following, coding, and math, and IBM's docs position the family in 3B, 8B, and 30B sizes with long-context training up to 512K. But Granite Switch is strategically more important for APEX than moving from one plain instruct size to another, because Granite Switch auto-composes the Granite RAG, Core, and Guardian adapters into one model surface. In practice, that means you could have replaced multiple brittle patches with purpose-built intrinsics for query rewrite, query clarification, answerability, context relevance, hallucination detection, citation generation, requirements validation, uncertainty/context attribution, and policy/factuality guardrails. That is a much better fit for a compliance-sensitive adaptive-racing coach than a generic narrator plus regex scrubbers and fallback UI text. ⁵

The deeper reason this matters is that APEX's real moat is **not** "we forecast curves." It is "we can explain a recommendation that is faithful to telemetry, grounded in the driver's adaptations, and auditable." Granite Docling is explicitly designed for structured enterprise document conversion with DocTags that preserve hierarchy, tables, and reading order for downstream RAG. Mellea is explicitly designed to turn unreliable LLM calls into typed, requirement-checked pipelines with instruct-validate-repair loops. watsonx Orchestrate is explicitly pushing agentic workflows, loop controls, Python toolkit integration, and MCP-based workflows. watsonx.governance is explicitly pushing monitoring, Evaluation Studio, factsheets, and agent-risk workflows. Your public stack gestures toward all of that, but the runtime facets a judge can verify are still only partway there. ⁶

Brutal judge criticisms

You should have made Granite Switch the control plane

A brutal judge could say: "**You used Granite like a general chatbot, when IBM's newest Granite release for exactly this kind of workflow was Granite Switch plus the Granite Libraries.**" That criticism would be fair. Your public chat route still has a stub phase, and even the production path is essentially a plain chat-completions proxy followed by a server-side regex scrubber for invented legal anchors. Granite Switch preview, by contrast, is built to combine Granite's RAG, Core, and Guardian intrinsics in one place, and Mellea's IVR loop is built to retry or repair outputs against explicit requirements. That lands harder than a plain narrator because it demonstrates reliability as a system property, not just a prompt aspiration. ⁷

Missed opportunity. You could have made "answerability before answer, citations per claim, policy check before surface, repair on failure" the center of the judging story instead of "we stream a Granite-flavored answer and scrub invented references afterward." ⁸

Easiest fix by May 31. Put Granite Switch 4.1-3B preview only on the **AICopilotChat and final report verification** path. Do not re-architect the whole app. One visible, live path using IBM's newest intrinsic composition story is enough to change the narrative. ⁹

Realistic 24–48 hour fix. Wrap the final coaching recommendation in a Mellea IVR loop with three requirements: every recommendation must be answerable from retrieved material, every factual sentence

must have a source citation, and every output must pass Guardian/policy validation before it is shown. That is exactly the sort of loop Mellea and the Granite Libraries were released to support. ¹⁰

You should have made real COA PDF understanding your centerpiece

A brutal judge could say: **“Your differentiator is adaptive-driver coaching, but your COA path in public code still starts from a JSON stub.”** That is dangerous because the COA is not a side input; it is the heart of the adaptation-specific coaching claim. Granite Docling is specifically designed for end-to-end document understanding and structured conversion, with DocTags meant to preserve the structure that makes downstream retrieval and citation work. The repo docs already present Granite-Docling as a core tool, but the public parser still says the live PDF path is a future swap-point. ¹¹

Missed opportunity. If you had converted a real COA PDF into structured JSON with preserved Appendix L references and then cited those clauses in the recommendation, you would have had a uniquely strong “adaptive motorsport compliance” demo. That is far more defensible than a generic “LLM plus charts” pitch. ¹²

Easiest fix by May 31. Narrow scope. Support **one known COA template** end to end with Granite-Docling and cache the result by file hash, exactly the way your architecture spec already frames it. A one-template live parse is more persuasive than a future-perfect parser claim. ¹³

Realistic 24–48 hour fix. Wire Granite-Docling plus the Docling library for the demo COA PDFs only, emit the existing `CoaParseResult` shape, and surface the extracted clause IDs directly in the UI footer and audit trace. You do not need universal PDF support to win this point with judges. ¹⁴

You should have shown a live workflow engine instead of orchestration theater

A brutal judge could say: **“Your orchestration story is a facade.”** The architecture spec says LangGraph + MCP + ContextForge is the runtime, but the public frontend panel itself says HEAD is showing a canned-fallback engine while the real runtime is still a swap-point. IBM’s watsonx Orchestrate story in late 2025 and May 2026 is very clear: reusable agentic workflows, Langflow integration, loop controls, Python toolkit integration, tool import through MCP, and richer traceability. APEX’s public runtime panel is still much closer to a demo shell than to that story. ¹⁵

Missed opportunity. In a challenge setting, a real typed workflow trace often scores better than an extra model. Judges can see control, retry logic, and tool boundaries. Right now your panel tells them the opposite: the serious runtime is still coming. ¹⁶

Easiest fix by May 31. Make `/api/orchestration` emit a **real trace from your current backend steps**, even if it is only six nodes and entirely local. Live traces beat canned traces. ¹⁷

Realistic 24–48 hour fix. Wire one real “retrieve → project → audit → narrate → provenance” workflow in LangGraph or watsonx Orchestrate, bundle the numeric tools as a Python toolkit-style unit, and expose the actual trace IDs, statuses, and timings in the judges panel. The May 2026 Orchestrate release makes that pattern especially current. ¹⁸

You should have turned Guardian into governance evidence

A brutal judge could say: **“A safety stamp is not governance.”** Granite Guardian 4.1’s upgraded BYOC support is specifically about evaluating complex, multi-part requirements, not just issuing a single verdict. The Granite Guardian library expands that pattern further into safety, factuality detection/correction, and policy guardrails. IBM’s watsonx.governance stack adds monitoring from request to production, Evaluation Studio for comparing generative assets, factsheets, and even new governance support for AI agents. In the public repo, though, the Guardian backend file still describes the live code as the deterministic floor the model will later wrap. That leaves an obvious gap between “audited” and “governed.” ¹⁹

Missed opportunity. APEX is making coaching recommendations in a physically constrained, accessibility-sensitive domain. That domain practically begs for a judge-facing governance artifact: criteria, failure cases, monitored thresholds, and model facts. Right now the public story is strong on rhetoric and weaker on governance evidence. ²⁰

Easiest fix by May 31. Run the final recommendation through actual Granite Guardian 4.1 BYOC or the Guardian intrinsics and export a one-page evaluation card: pass/fail counts, top rule families, blocked recommendation examples, and hallucination/factuality status. ²¹

Realistic 24–48 hour fix. If you have watsonx.governance access, push the prompt templates or agent surfaces into Evaluation Studio and produce comparative outputs and a factsheet-like artifact. If you do not, mimic the same evidence model locally: evaluation set, thresholds, logs, and traceable results. Either way, show governance as an artifact, not an adjective. ²²

You should have replaced browser speech with Granite Speech

A brutal judge could say: **“You solved the easiest version of voice input.”** The public VoiceDebriefInput component states plainly that HEAD uses browser-native SpeechRecognition and that Watson STT is still a queued swap-point, while the public server route is TTS, not STT. IBM Granite Speech 4.1 2B was released for multilingual ASR and speech translation; the 2B Plus model adds speaker-attributed transcription and word-level timing; the 2B NAR variant is aimed at latency-sensitive ASR. For APEX specifically, timestamps are not a nicety—they are the bridge between a spoken debrief and telemetry sectors. ²³

Missed opportunity. A real Granite Speech integration would let you align phrases like “late turn-in here” or “I hesitated on pickup” to time windows and mini-sectors. That is a deeply more convincing coaching loop than a raw browser transcript pasted into a text box. ²⁴

Easiest fix by May 31. Add a simple “record/upload audio” path and run batch transcription on the backend with Granite Speech 4.1 2B. You do not need streaming to make the demo materially stronger. ²⁵

Realistic 24–48 hour fix. Use Granite Speech 4.1 2B Plus if you want word timestamps for alignment, or 2B NAR if you care more about low-latency turnaround. Then map those timestamps against sector windows so the debrief can literally light up the relevant part of the lap. ²⁶

Direct answers on variants, code, speech, and embeddings

Granite variant you should have used but did not. The strongest answer is **Granite Switch 4.1-3B preview**. Not because 3B is “better” than 8B in the abstract, but because the Switch model composition solves the actual weakness in APEX: missing intrinsics for answerability, citations, policy checks, and repair. I would **not** burn deadline time chasing a plain 30B narrator. Granite 4.1 already improved tool calling and instruction following across the family; the architectural delta that matters for you is *intrinsic*s, not raw parameter count. ²⁷

Granite tool you are using shallowly. The shallowest usage is a tie between **Granite Guardian** and **Granite Embedding**. Guardian is public-facing in the repo, but the public backend file still frames the live code as the deterministic floor rather than the full model integration. Embedding is also currently underexploited because IBM’s own Embedding docs treat the reranker as a first-class way to improve retrieval quality, and Granite’s RAG library adds pre-retrieval and post-generation intrinsics like query rewrite, answerability, hallucination detection, and citation generation. If you want one sentence: you are using IBM models more like components in a diagram than as a tightly coupled retrieval-and-verification system. ²⁸

Pattern from watsonx Orchestrate, watsonx.governance, Mellea, or Granite agentic flows that would land harder. The most compelling pattern is: **typed workflow → requirement checks → repair loop → governance artifact**. Mellea gives you instruct-validate-repair and constrained structure. Granite Libraries and Switch give you the intrinsics. watsonx Orchestrate gives you reusable workflows, tracing, loop controls, and Python toolkit bundling. watsonx.governance gives you comparison, monitoring, and factual/governance evidence. That combination would feel much more “IBM agentic system” than the current public mix of panels, swap-points, and route stubs. ²⁹

Granite Code variant for an explain-your-coaching-recommendation-in-Python feature. Yes: use **Granite Code 8B Instruct 128K** if you want a serious “show me the NumPy/Matplotlib code that reproduces this recommendation” feature. IBM’s Granite Code docs say Code models are available in 3B, 8B, 20B, and 34B sizes, and the 8B 128K instruct model is intended for coding assistants over long-context inputs. If you need a lighter or more local path, the older 8B Code Instruct 4K model also exists, including GGUF conversions. For your use case, 8B is the sweet spot between capability and deadline risk. ³⁰

Granite Speech model to replace the current Web Speech API voice-debrief. Yes. Use **Granite Speech 4.1 2B** for the straightforward replacement, **Granite Speech 4.1 2B Plus** if you want timestamp-rich transcripts or speaker labels, and **Granite Speech 4.1 2B NAR** if latency is the dominant concern. For APEX, I would choose **2B Plus** unless you are desperate for speed, because the timestamping creates a much better story with telemetry alignment. ³¹

Granite Embedding tier you should have used. The answer is **split-tier, not total replacement**. Keep **Granite-embedding-english-r2 149M** on the server where you care about relevance quality and use the Granite reranker after retrieval. For edge or offline surfaces, use **Granite-embedding-30m-english**; it is a 30M bi-encoder with 384-dimensional vectors and is much more appropriate for small-footprint retrieval than a 149M server encoder. If you eventually need multilingual support, IBM’s current multilingual R2 family is 97M and 311M. So: yes, 30M is the right edge tier; no, I would not demote the whole system to 30M right before submission. ³²

Final ranking

If you only land **one** additional IBM move before May 31, land **Granite Switch + Mellea IVR** on the narration and chat path. That is the highest-leverage move because it converts the current public weak point—generic output control—into IBM's most current reliability story. If you can land **two**, make the **COA PDF path real with Granite-Docling**. If you can land **three**, choose between **Granite Speech with timestamps** and a **real workflow trace** depending on whether your judges care more about accessibility interaction or agentic architecture. Do **not** lose the week swapping to a bigger plain LLM. The public repo does not have a model-size problem. It has a **depth-of-integration problem**. ³³

¹ ⁷ <https://github.com/StephenSook/apex/raw/refs/heads/main/app/frontend/app/api/openrouter-stream/route.ts>

<https://github.com/StephenSook/apex/raw/refs/heads/main/app/frontend/app/api/openrouter-stream/route.ts>

² ⁴ ²⁰ [GitHub - StephenSook/apex: AI race engineer for adaptive racers, built on IBM Granite. IBM SkillsBuild May 2026. · GitHub](#)

<https://github.com/StephenSook/apex>

³ https://github.com/StephenSook/apex/raw/refs/heads/main/app/backend/apex/instruct/coa_parser.py

https://github.com/StephenSook/apex/raw/refs/heads/main/app/backend/apex/instruct/coa_parser.py

⁵ ²⁷ [Granite 4.1](#)

https://www.ibm.com/granite/docs/models/granite4-1?utm_source=chatgpt.com

⁶ ¹¹ ¹² [Granite Docling](#)

https://www.ibm.com/granite/docs/models/docling?utm_source=chatgpt.com

⁸ [ibm-granite/granitelib-rag-r1.0](#)

https://huggingface.co/ibm-granite/granitelib-rag-r1.0?utm_source=chatgpt.com

⁹ ³³ <https://huggingface.co/ibm-granite/granite-switch-4.1-3b-preview>

<https://huggingface.co/ibm-granite/granite-switch-4.1-3b-preview>

¹⁰ ²⁹ [The Instruction Model - Mellea](#)

https://docs.mellea.ai/concepts/instruct-validate-repair?utm_source=chatgpt.com

¹³ ¹⁴ ¹⁵ [apex/docs/architecture-spec.md at main · StephenSook/apex · GitHub](#)

<https://github.com/StephenSook/apex/blob/main/docs/architecture-spec.md>

¹⁶ ¹⁷ <https://github.com/StephenSook/apex/raw/refs/heads/main/app/frontend/components/LangGraphRuntimePanel.tsx>

<https://github.com/StephenSook/apex/raw/refs/heads/main/app/frontend/components/LangGraphRuntimePanel.tsx>

¹⁸ [What's new in IBM® watsonx Orchestrate®](#)

https://www.ibm.com/docs/en/watsonx/watson-orchestrate/base?topic=notes-whats-new&utm_source=chatgpt.com

¹⁹ ²¹ [Granite Guardian 4.1](#)

https://www.ibm.com/granite/docs/models/guardian?utm_source=chatgpt.com

²² [Governing assets with watsonx.governance](#)

https://www.ibm.com/docs/en/watsonx/saas?topic=governing-ai&utm_source=chatgpt.com

23 <https://github.com/StephenSook/apex/raw/refs/heads/main/app/frontend/components/VoiceDebriefInput.tsx>

<https://github.com/StephenSook/apex/raw/refs/heads/main/app/frontend/components/VoiceDebriefInput.tsx>

24 26 **ibm-granite/granite-speech-4.1-2b-plus**

https://huggingface.co/ibm-granite/granite-speech-4.1-2b-plus?utm_source=chatgpt.com

25 31 **ibm-granite/granite-speech-4.1-2b**

https://huggingface.co/ibm-granite/granite-speech-4.1-2b?utm_source=chatgpt.com

28 <https://github.com/StephenSook/apex/raw/refs/heads/main/app/backend/apex/guardian/audit.py>

<https://github.com/StephenSook/apex/raw/refs/heads/main/app/backend/apex/guardian/audit.py>

30 **Granite Code**

https://www.ibm.com/granite/docs/models/code?utm_source=chatgpt.com

32 **ibm-granite/granite-embedding-english-r2**

https://huggingface.co/ibm-granite/granite-embedding-english-r2?utm_source=chatgpt.com